

Recursion

Announcements

Quiz Debugging

Environment Diagram Review

Code & Environments

Code can refer to local names, enclosing function names, and global names

For any **expression**, you know the frames it will be **evaluated in**
without drawing the whole environment diagram

- For any expression that appears outside a **def** statement or **lambda** expression, the environment is just the global frame
- For other expressions: the environment has a frame for every enclosing **def** statement or **lambda** expression, ordered from inner to outer

```
def exp(x):  
    def base(b):  
        return pow(b, x)  
    return base
```

b will be found
in a **base** frame

x will be found in an **exp** frame:
the argument to an **exp** call

Will always be evaluated in an
environment with 3 frames:
base, **exp**, and then **global**

f1:exp	parent:Global
x	_____
Return value	_____

f2:base	parent:f1
b	_____
Return value	_____

For any **name**, you know the frame it will be **found in**
without drawing the whole environment diagram

What is printed by this code?
(Fall 2026 CS 61A Midterm 1)

```
def new_print(print):  
    def f(x):  
        value = print(x)  
        if value != None:  
            return print('What?')  
        return x  
    og = print  
    print = new_print(print)  
    f whose print is the original print  
    print = new_print(print)  
    og('print returned', print(2))
```

print will be found in a new_print frame

if value != None:

return print('What?')

return x

also here!

Eval'd in some f, new_print, Global env

original print

f whose print is the original print

print = new_print(print)

og('print returned', print(2))

f whose print is (f whose print is the original print)

Global frame	<u>new_print</u>	<u> </u>
What is called print	<u> </u>	<u> </u>
og	<u> </u>	<u> </u>

f1: <u>new_print</u>	[parent= <u>Global</u>]	
<u>print</u>	<u> </u>	<u> </u>
<u>f</u>	<u> </u>	<u> </u>
Return Value	<u> </u>	<u> </u>

f2: <u>new_print</u>	[parent= <u>Global</u>]	
<u>print</u>	<u> </u>	<u> </u>
<u>f</u>	<u> </u>	<u> </u>
Return Value	<u> </u>	<u> </u>

f3: <u>f</u>	[parent= <u>f2</u>]	
<u>x</u>	<u>2</u>	<u> </u>
<u>value</u>	<u>2</u>	<u> </u>
Return Value	<u>'What'</u>	<u> </u>

f4: <u>f</u>	[parent= <u>f1</u>]	
<u>x</u>	<u>2</u>	<u> </u>
<u>value</u>	<u>None</u>	<u> </u>
Return Value	<u>2</u>	<u> </u>

f5: <u>f</u>	[parent= <u>f1</u>]	
<u>x</u>	<u>'What'</u>	<u> </u>
<u>value</u>	<u>None</u>	<u> </u>
Return Value	<u>'What'</u>	<u> </u>

func new_print(print) [parent=G]

func print(...) the original!

func f(x) whose print is original

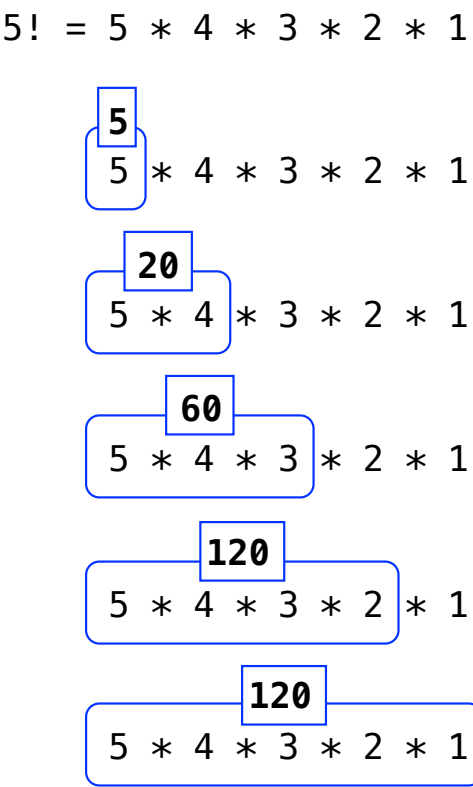
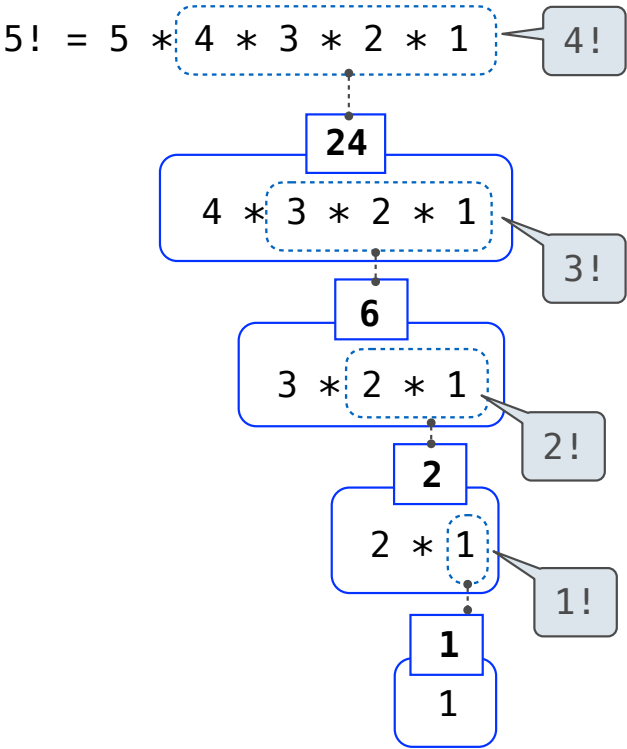
func f(x) whose print is

Printed output

2
What?
print returned What?

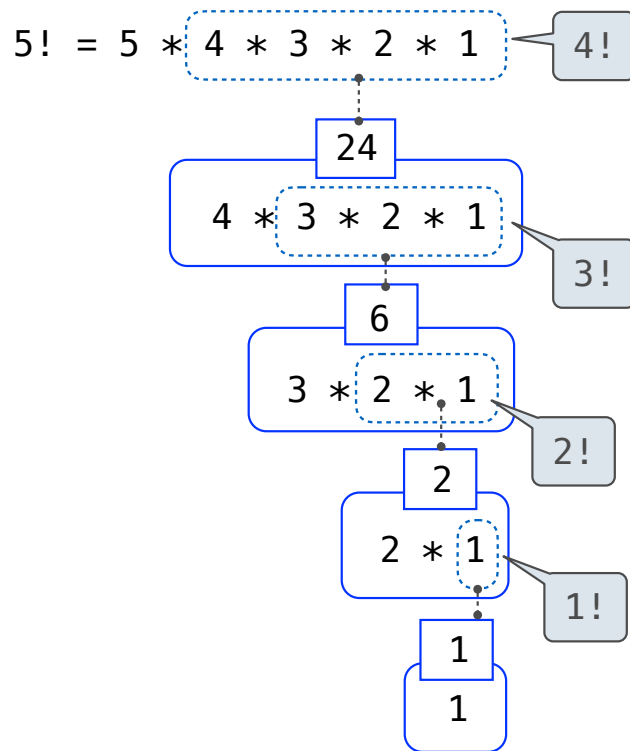
Recursive Functions

Factorial

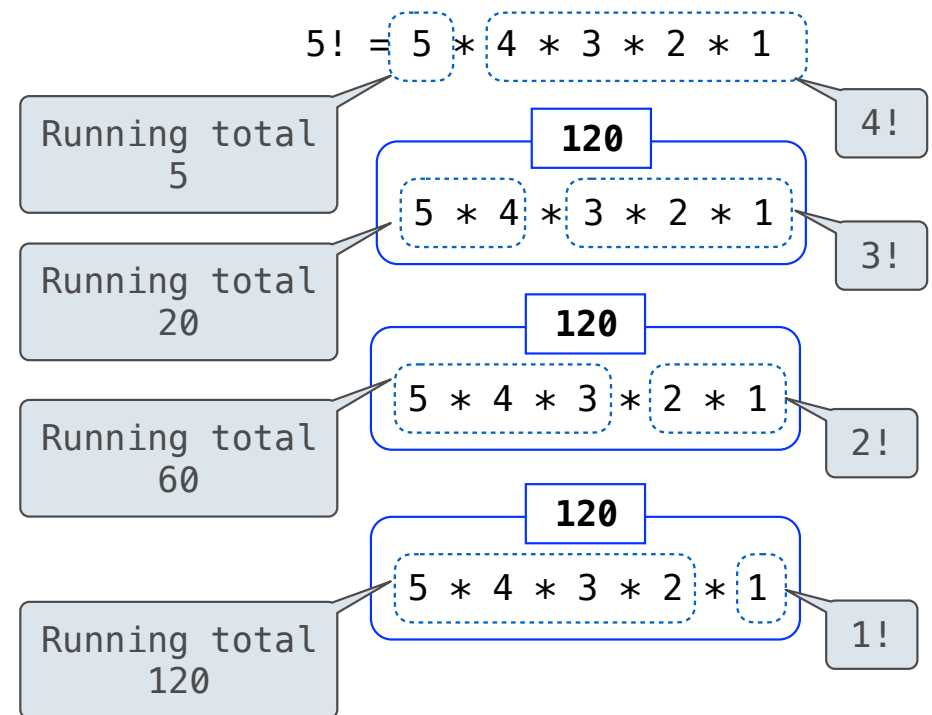


Factorial

Recursive approach, version 1:



while loop, fact_tail:



Converting Iteration to Recursion

Idea: The state of the while loop is passed as arguments

Factorial

Any iterative function can be written recursively

Idea: The changing state of the while loop can be passed as arguments instead

```
def fact_while(n):  
    result = 1  
    while n > 0:  
        result = result * n  
        n -= 1  
    return result
```

```
def fact(n):  
    def f(n, result):  
        if not n > 0:  
            return result  
        result = result * n  
        n -= 1  
        return f(n, result)  
    return f(n, 1)
```

Twenty-One Rules

Two players alternate turns, on which they can add 1, 2, or 3 to the current total

The total starts at 0

The game end whenever the total is 21 or more

The last player to add to the total loses

Discussion Question: Play Twenty-One

Rewrite play as a recursive function without a while statement.

- Do you need to define a new inner function? Why or why not? If so, what are its arguments?
- What is the base case and what is returned for the base case?

```
def play(strategy0, strategy1, goal=21):  
    """Play twenty-one and return the winner.
```

```
>>> play(some_strat, some_other_strat)
```

```
1
```

```
"""
```

```
n = 0
```

```
who = 0 # Player 0 goes first
```

```
while n < goal:
```

```
    if who == 0:
```

```
        n = n + strategy0(n)
```

```
        who = 1
```

```
    elif who == 1:
```

```
        n = n + strategy1(n)
```

```
        who = 0
```

```
return who
```

```
def play(strategy0, strategy1, goal=21):  
    """Play twenty-one and return the winner.
```

```
>>> play(some_strat, some_other_strat)
```

```
1
```

```
"""
```

```
def f(n, who):
```

```
    if n >= goal:
```

```
        return who
```

```
    if who == 0:
```

```
        n = n + strategy0(n)
```

```
        who = 1
```

```
    elif who == 1:
```

```
        n = n + strategy1(n)
```

```
        who = 0
```

```
    return f(n, who)
```

```
return f(0, 0)
```